

VENTAS PARA FREELANCERS Y DEVELOPERS

Venta consultiva: como vender sin sentir que vendés

Para developers y freelancers que resuelven bien lo tecnico pero se estancan a la hora de conseguir y cerrar clientes. Como hacer las preguntas correctas antes de ofrecer nada.

Discovery calls

SPIN Selling

Manejo de objeciones

Índice

- 01 Introducción — por que los developers venden mal
- 02 El mindset: vender es diagnosticar, no convencer
- 03 Antes de la llamada: investigar al prospecto
- 04 El framework SPIN, adaptado a lenguaje tecnico
- 05 La estructura de la discovery call, paso a paso
- 06 Errores comunes de developers vendiendo
- 07 De la llamada a la propuesta, sin que parezca un pitch
- 08 Objeciones comunes y checklist final

01 · Introducción

Si desarrollás bien pero sentís que estás estancado, lo más probable es que el problema no sea técnico. Es que nadie te enseñó a vender — y peor, a nadie le gusta la idea de "vender", así que directamente lo evitan.

Los developers tienden a cometer siempre el mismo error: apenas alguien menciona un problema, saltan directo a la solución técnica. "Necesito una web" se convierte en un presupuesto de funcionalidades antes de entender para qué la necesita esa persona, qué pasa hoy sin ella, o cuánto vale realmente resolverlo. Esto no es un problema de habilidad técnica — es un problema de **secuencia**: se ofrece antes de diagnosticar.

Esta guía no es un curso de "técnicas de cierre" ni de discursos memorizados. Es un método para hacer las preguntas correctas, en el orden correcto, antes de ofrecer nada — lo que en ventas se llama **venta consultiva**. No se trata de convencer a nadie de nada: se trata de entender tan bien el problema del cliente que la solución se vuelve obvia para los dos.

QUÉ VAS A PODER HACER AL TERMINAR ESTA GUÍA

Encarar una primera llamada con un cliente potencial sin improvisar, hacer preguntas que revelan el problema real (no el que el cliente cree tener), calificar si vale la pena seguir antes de cotizar, y pasar de la conversación a una propuesta sin que se sienta un pitch forzado.

02 · El mindset: vender es diagnosticar

El cambio mas importante de esta guia no es una tecnica, es una forma distinta de pensar la primera conversacion con un cliente.

Vender, en el sentido en que lo vamos a usar aca, no es persuadir a alguien de comprar algo que no necesita. Es mas parecido a lo que ya sabes hacer todo el tiempo como developer:

diagnosticar antes de proponer una solucion. Nadie empieza a programar sin entender primero que tiene que resolver el codigo — la primera llamada con un cliente deberia funcionar exactamente igual.

Una **discovery call** (llamada de descubrimiento) es, literalmente, eso: una conversacion diagnostica, no un pitch de ventas disfrazado. El objetivo no es convencer a la otra persona de que la contrate — es entender si hay un problema real, si vos sos la persona correcta para resolverlo, y si tiene sentido seguir adelante para los dos. A veces la respuesta honesta es que no.

POR QUE ESTE CAMBIO DE MENTALIDAD IMPORTA

Cuando dejás de pensar "tengo que convencerlo" y empezas a pensar "tengo que entender si esto tiene sentido", la conversacion se relaja para los dos lados — y paradójicamente, es mucho mas efectiva para cerrar proyectos que valen la pena.

03 · Antes de la llamada: investigar al prospecto

La calidad de tus preguntas depende de cuanto sabes antes de empezar a preguntar.

Diez minutos de investigacion previa cambian por completo una discovery call. Antes de la llamada, conviene tener claro:

- **Que hace la empresa o la persona**, y en que industria opera.
- **Como llegaron a vos** — recomendacion, contenido que publicaste, busqueda directa. Esto te dice que tan calificado viene el contacto.
- **Cualquier contexto publico relevante** — su web actual, su presencia online, senales de que estan creciendo o de que algo especifico dispar esta necesidad ahora.

No se trata de investigar por investigar — se trata de no hacerle perder tiempo a nadie preguntando cosas que ya podrias saber, y de poder hacer preguntas mas especificas y relevantes desde el minuto uno.

Ojo: investigar de mas y llegar con ideas ya armadas de "la solucion" es tan malo como no investigar nada — te hace saltar directo a proponer antes de escuchar. La investigacion previa es para hacer mejores preguntas, no para reemplazarlas.

04 · El framework SPIN, adaptado a lenguaje técnico

SPIN es el framework de venta consultiva mas estudiado y usado desde hace decadas. Sus cuatro tipos de pregunta, en orden, llevan la conversacion de "cuentame que pasa" a "necesito que me ayudes a resolver esto".

S — Situación

Entender el contexto actual, sin asumir nada.

GUION DE EJEMPLO

Contame como resuelven esto hoy. ¿Que herramientas o proceso usan actualmente?

P — Problema

Identificar friccion o dificultades concretas en esa situacion actual.

GUION DE EJEMPLO

¿Que es lo que mas les cuesta o les hace perder tiempo de esa forma de trabajar hoy?

I — Implicación

Ampliar el costo real de no resolver ese problema — esta es la pregunta que mas suelen saltarse los developers, y la mas importante.

GUION DE EJEMPLO

Si esto sigue igual los proximos seis meses, ¿que impacto tiene eso en el resto del equipo o del negocio?

N — Necesidad-beneficio

Hacer que sea el cliente quien articule el valor de resolverlo — no vos.

GUION DE EJEMPLO

Si esto se resolviera, ¿que cambiaria concretamente para ustedes?

POR QUE FUNCIONA

Cuando el cliente es quien pone en palabras el costo del problema y el valor de resolverlo, ya no hace falta que vos lo convenzas de nada — el mismo llega a esa conclusion.

05 · La estructura de la discovery call, paso a paso

Una discovery call de 30 minutos, con un objetivo claro en cada tramo.

1 Rapport (2-3 min)

Romper el hielo brevemente, sin alargarlo — el objetivo es que la otra persona se sienta cómoda, no llenar tiempo.

2 Contexto y confirmacion (3-5 min)

Confirmar brevemente lo que ya investigaste y dejar que la otra persona corrija o agregue contexto.

3 Descubrimiento con SPIN (12-15 min)

El nucleo de la llamada: Situacion, Problema, Implicacion, Necesidad-beneficio, en ese orden. Aca es donde se hacen la mayoría de las preguntas.

4 Calificar (5 min)

Confirmar presupuesto aproximado, quien toma la decision final, y los plazos reales — sin esto, se puede armar una propuesta perfecta para alguien que nunca va a poder avanzar.

5 Cierre y proximos pasos (3-5 min)

Nunca terminar la llamada sin un proximo paso concreto y una fecha: "te mando la propuesta el jueves" es mucho mejor que "despues te escribo".

GUION UTIL PARA CALIFICAR PRESUPUESTO SIN SONAR INVASIVO

Para armarte una propuesta que tenga sentido, ¿tienen ya un rango de presupuesto pensado para esto, o todavia lo estan evaluando?

06 · Errores comunes de developers vendiendo

- **Saltar directo a la solución técnica.** Empezar a hablar de stack, arquitectura o funcionalidades antes de haber entendido el problema real es la forma más rápida de perder el control de la conversación.
- **Cotizar en la primera llamada.** Dar un número sin haber calificado presupuesto ni entendido el alcance completo casi siempre termina mal — o cotizas de más, o de menos, y en ambos casos pierdes.
- **No calificar quien decide.** Armar una propuesta detallada para alguien que después tiene que "consultarlo con su socio" duplica el trabajo y diluye el impulso de la conversación.
- **Sobre-explicar la parte técnica.** El cliente casi nunca necesita entender cómo funciona la solución por dentro — necesita entender que su problema se va a resolver.
- **No preguntar por la implicación del problema.** Sin esa pregunta, el cliente nunca termina de sentir la urgencia real de resolverlo ahora.

07 · De la llamada a la propuesta

La transición de la discovery call a la propuesta formal es donde más se nota si la llamada anterior estuvo bien hecha.

Si el descubrimiento fue bueno, la propuesta no debería sonar a pitch — debería sonar a resumen. La estructura más efectiva es simple:

1. **Reflejar el problema con sus propias palabras.** Repetir lo que el cliente dijo sobre su situación y su problema, para confirmar que escuchaste bien.
2. **Conectar ese problema con la implicación que el mismo describió.** El costo de no resolverlo, en sus propias palabras.
3. **Presentar la solución como consecuencia lógica,** no como una lista de funcionalidades sueltas.
4. **Cerrar con un próximo paso claro,** no con un "avisame que te parece".

GUION DE APERTURA PARA LA PROPUESTA

Como charlamos, hoy [resumen breve de la situación]. Eso te está generando [problema/implicación que el cliente mencionó]. Te armé una propuesta pensada específicamente para resolver eso.

08 · Objeciones comunes y checklist final

"Es muy caro"

Rara vez es sobre el precio en si — casi siempre es sobre el valor percibido. Volver a la implicacion que el mismo cliente describio suele ser mas efectivo que bajar el precio.

GUION DE EJEMPLO

Entiendo. Volviendo a lo que me contaste antes, ¿como estarias pensando resolver esto si no es conmigo?

"Lo puedo hacer con IA / no-code"

Es una objecion cada vez mas comun. No conviene entrar en la defensiva — conviene volver al problema de fondo (tiempo, criterio, mantenimiento) mas que a la tecnologia en si.

GUION DE EJEMPLO

Totalmente valido. ¿Que tanto tiempo tenes disponible vos para armarlo y despues mantenerlo funcionando?

"Necesito pensarlo"

Suele significar que quedo alguna duda sin resolver, no necesariamente un no. Preguntar que falta es mejor que insistir.

GUION DE EJEMPLO

Por supuesto. ¿Hay algo puntual que te quedo dando vueltas, para que pueda aclararlo ahora?

- ✓ Investigaste al prospecto antes de la llamada.
- ✓ Hiciste preguntas de Situacion, Problema, Implicacion y Necesidad-beneficio, en ese orden.
- ✓ Calificaste presupuesto, quien decide, y los plazos.
- ✓ Terminaste la llamada con un proximo paso concreto y una fecha.
- ✓ Tu propuesta refleja las palabras del cliente, no una lista generica de funcionalidades.